Asset Management Transformation at a Regulated Electricity Distributor

Nov 5, 2026 · @Ojowu Omakwu

Business architecture · Enterprise asset management · Integration analysis · Adoption

Overview

I contributed to an enterprise asset management transformation at a regulated electricity distributor. The organization was moving from fragmented asset information and manual processes to a coordinated approach built on IBM Maximo, integrated with GIS and SAP.

My work was to understand the business processes the technology had to support, define the future-state capabilities and workflows, and help stakeholders decide on sequencing, integration requirements and adoption. The first phase delivered core asset information and inspection capability. Where this account describes operating-model options, they were my recommendations, not changes I claim were all implemented.

This account is deliberately general. It describes my responsibilities and reasoning, not the organization's internal documentation or designs.

The business challenge

Asset, inspection, work and financial information was spread across several tools and systems. That made it hard to keep information consistent, coordinate work across functions, and rely on the data for planning and reporting. New software alone would not fix the underlying process and coordination problems.

So the real question was broader than choosing a platform: which capabilities and ways of working had to change for the investment to improve how work was planned, managed and recorded?

My responsibility

I was the business analyst on a small core internal IT team, reporting to the senior manager.

I led	I contributed to	Others owned
Value stream and capability mapping; process analysis; future-state operating-model design; the business case; requirements; process and integration specifications; data mapping	Integration solution design, working with technical specialists on design trade-offs; ADKAR-informed stakeholder engagement and readiness	Platform configuration and integration development

Approach and professional judgment

1. Find the real constraint before designing the solution

I mapped the works delivery value stream across functional boundaries and assessed each stage for timeliness, efficiency and effectiveness. The constraint was not where the organization expected it. It sat in planning and design, upstream of field delivery, not in field capacity. Framing the problem around that finding moved the conversation from each department's requirements to the outcome they shared.

2. Challenge an established position with evidence

The hardest moment was questioning an approach stakeholders believed was the answer: a detailed works program fixed a year in advance, intended to give delivery certainty. The evidence suggested it did not address the constraint. Contractors typically schedule crews only a couple of months ahead, so a detailed annual list goes stale long before most of it is delivered.

I didn't argue that the existing approach was wrong. I used the process evidence to make the constraint visible, then recommended an alternative: an annual plan at forecast level, specific work committed in shorter rolling cycles, more standardization of routine work, and clearer coordination between planning and delivery. It balanced planning certainty against the flexibility delivery needed.

What I took from it: challenging a position works far better when you anchor on the shared outcome, bring evidence, and offer a practical alternative instead of only pointing out what isn't working.

3. Sequence capabilities by dependency and value

Each function had legitimate but competing priorities for the new platform, and taking them all on at once would have added complexity, dependencies and delivery risk. This was an operating-model decision more than a requirements one. Using capability gaps, end-to-end processes and cross-functional dependencies, we shaped a phased approach. The first rollout covered inspection and core maintenance, which were foundational and carried most of the volume. More complex functions followed in later phases.

4. Define integration requirements and data ownership

I wrote the integration specification and data mappings between asset management and geospatial information, covering which system owns which data, record matching, consistency and exception handling. I contributed to the evaluation of solution options with the technical team. The first phase delivered asset-data synchronization with GIS; financial integration was planned for a later phase.

5. Plan for adoption, not just deployment

The change affected how teams captured inspections, managed information and coordinated work. I used ADKAR-informed engagement, with the process evidence building the case for change, to support awareness, readiness and adoption. Phasing helped too: teams saw improvement early instead of living through one long change.

Outcomes

- Core asset information was brought into Maximo and kept aligned with the GIS source.
- Inspection records were linked to the assets they describe.
- Future-state processes reduced reliance on spreadsheet handoffs and disconnected records.

No formal baseline was set before the change, so benefits could be observed but not cleanly attributed to the program. That gap became one of my main lessons.

Lessons, and where they went next

The biggest lesson was that the value of enterprise technology depends on operating-model decisions: finding the real constraint, aligning capabilities, giving data clear owners, and sequencing change around readiness and dependencies. I now apply these lessons in my AI transformation portfolio, which follows NorthRiver, a fictional distributor, through its first AI use case. The case is my independent work.

Lesson from this project	Where it shows up in the portfolio
Find the real constraint before choosing a solution	The portfolio starts from an AI adoption value stream and scores use cases on evidence before any build
Sequence by dependency, not by who asks loudest	The three-wave capability roadmap separates investment priority from sequence, and shows where dependency overrides priority
Without a baseline, benefits can only be estimated	The business case and adoption playbook require a timed baseline with the pilot group before anyone sees the tool
The operating model matters as much as the platform	The playbook treats workflow change and role impacts as part of the design, planned before build
When data can't be matched, send it to a person rather than guess	The knowledge assistant must cite its sources and decline when the evidence is missing